EP - Python Scientific Computing & Basic Plotting (Numpy & Matplotlib)

In this lab, we will study a new library in python that offers fast, memory efficient manipulation of vectors, matrices and tensors: **numpy**. We will also study basic plotting of data using the most popular data visualization libraries in the python ecosystem: **matplotlib**.

```
# Some IPython magic
# Put these at the top of every notebook, to get automatic reloading and inline plotting
%reload_ext autoreload
%autoreload 2

import numpy as np
```

Python scientific computing ecosystem

For scientific computing we need an environment that is easy to use, and provides a couple of tools like manipulating data and visualizing results. Python is very easy to use, but the downside is that it's not fast at numerical computing. Luckily, we have very eficient libraries for all our use-cases.

Libraries

Core computing libraries

- `numpy` and `scipy`: scientific computing
- `matplotlib`: plotting library

Machine Learning

- `sklearn`: machine learning toolkit
- `tensorflow`: deep learning framework developed by google
- `keras`: deep learning framework on top of `tensorflow` for easier implementation
- `pytorch`: deep learning framework developed by facebook

Statistics and data analysis

- `pandas`: very popular data analysis library
- `statsmodels`: statistics

We also have advanced interactive environments:

- Ipython: advanced python console
- Jupyter: notebooks in the browser

There are many more scientific libraries available.

Check out these cheetsheets for fast reference to the common libraries:

Cheat sheets:

- python
- numpy
- matplotlib
- sklearn
- pandas

Other:

- Probabilities & Stats Refresher
- Algebra

numpy

`numpy` works with tensors of data, the main data structure is `numpy.array` or `numpy.ndarray`.

Why it is useful: Memory-efficient container that provides fast numerical operations.

Speed test

```
%timeit [x**2 for x in range(1000)]

37.7 µs ± 4.15 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)

%timeit np.arange(1000)**2

4.06 µs ± 430 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)
```

It is clear that math operations using numpy arrays are far more efficient computation-wise than using plain python lists. More than that, numpy arrays offer a rich API for ***tensor*** manipulation.

Working with numpy arrays

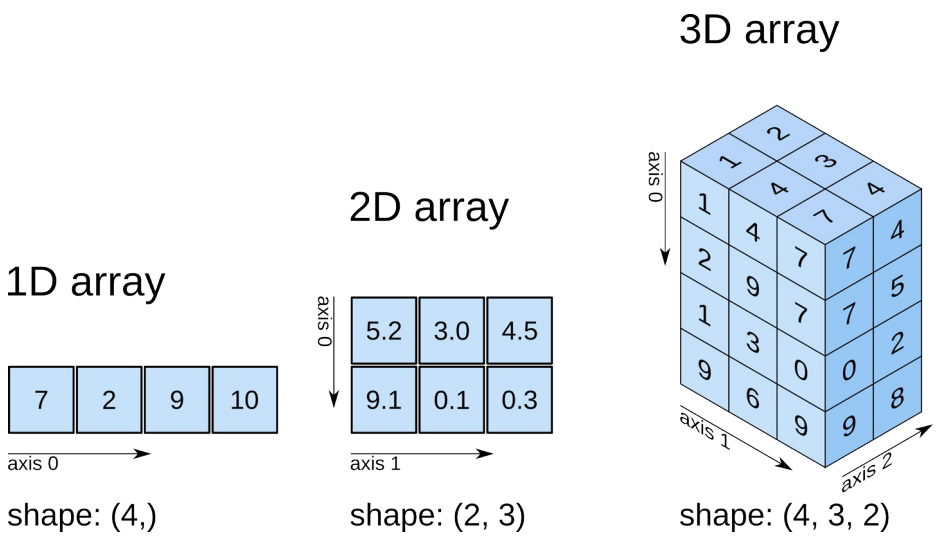
a few important attributes of the `array` structure:

- `ndarray.ndim` - number of axes (tensor rank)
- `ndarray.shape` - tuple, gives dimensions of each axes.
- `ndarray.size` - size of the array (product of elements of `ndarray.shape`)
- `ndarray.dtype` - data type

```
a = np.array([[2,7,5],[0,-1,2]])
print(a)
print("Dim:  ",a.ndim)
print("Shape:",a.shape)
print("Size: ",a.size)
print("Type: ",a.dtype)
```

```
[[ 2  7  5]
 [ 0 -1  2]]
Dim:   2
Shape: (2, 3)
Size:  6
Type:  int64
```

Shape Manipulation




```
x = np.array([1,1,2])
x.shape
```

```
(3,)
```

```
x.reshape(1,3)
```

```
array([[1, 1, 2]])
```

```
x.reshape(1,4) # this will fail because dimensions don't match
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-10-72e7449de1fa> in <cell line: 0>()
----> 1 x.reshape(1,4) # this will fail because dimensions don't match

ValueError: cannot reshape array of size 3 into shape (1,4)
```

```
x.reshape(1,-1)
```

```
array([[1, 1, 2]])
```

```
a = np.floor(10*np.random.random((3,4))) # some random matrix
a
```

```
array([[1., 4., 5., 4.],
       [5., 8., 2., 9.],
       [5., 5., 9., 7.]])
```

```
a.shape # a tuple that describes the shape of the object
```

```
(3, 4)
```

```
a.ravel() # flattened list

array([1., 4., 5., 4., 5., 8., 2., 9., 5., 5., 9., 7.])

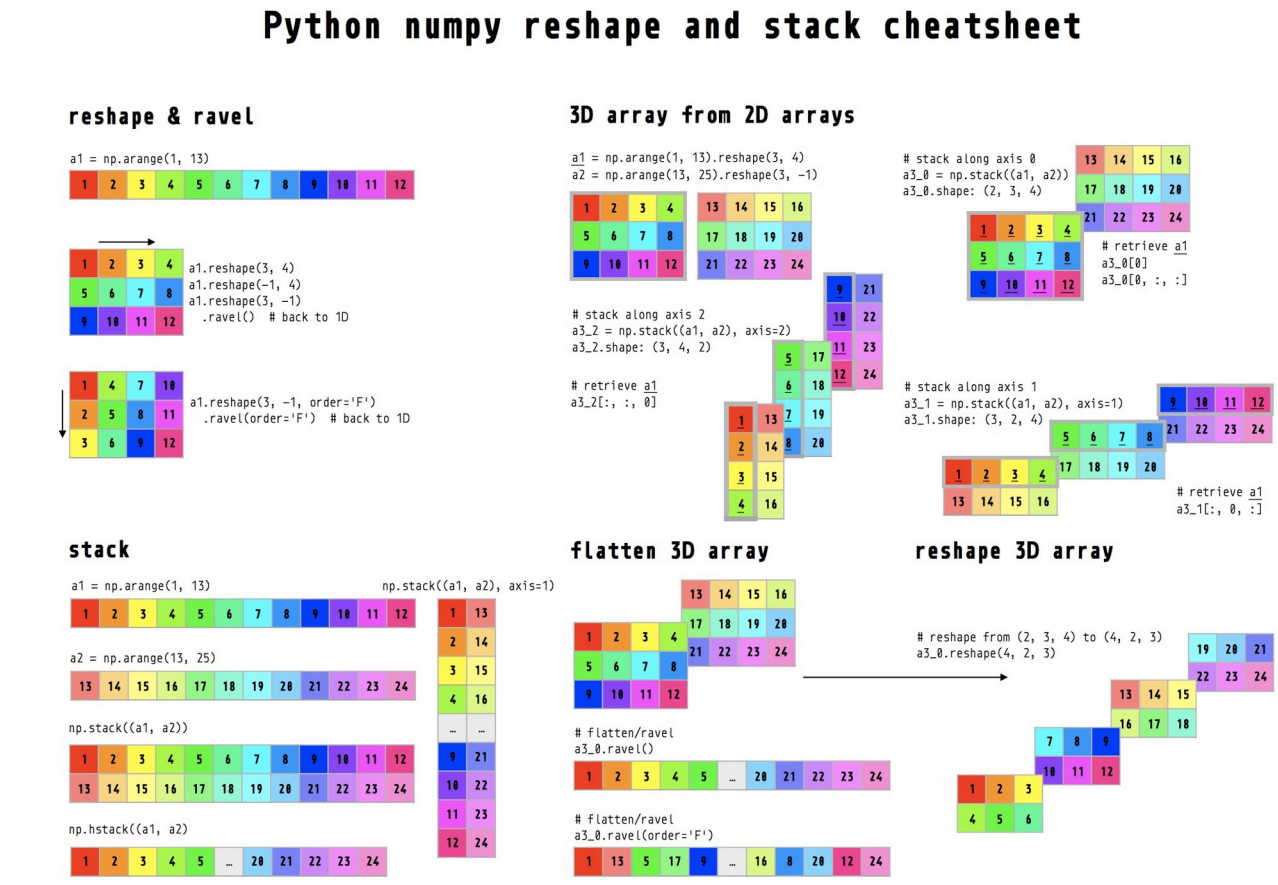
a.T # transpose

array([[1., 5., 5.],
       [4., 8., 5.],
       [5., 2., 9.],
       [4., 9., 7.]])

a.reshape(2,1,-1) # -1 means "as much as you need", such that the dimensions maches the size

array([[[1., 4., 5., 4., 5., 8.]],

       [[2., 9., 5., 5., 9., 7.]]])
```



Initializing

- `np.zeros(shape)`
- `np.ones(shape)`
- `np.empty(shape)`

- `np.eye(rows, cols)`

Those functions take as input the shape of an array.

Sequences

- `np.arange` - like Python's `range()`, but returns a `np.array`
- `np.linspace` - n equidistant points in interval [a,b]

```
np.arange(5)

array([0, 1, 2, 3, 4])

np.linspace(1, 2, 5)

array([1.  , 1.25, 1.5 , 1.75, 2.  ])
```

Math operators

Math operators are applied elementwise. (`*` is never the dot product for numpy arrays!)

```
a = np.ones((2,3), dtype=int)
a *= 3
a

array([[3, 3, 3],
       [3, 3, 3]])

a = a - 1
a

array([[2, 2, 2],
       [2, 2, 2]])

a = a + np.ones_like(a) / 5
a

array([[2.2, 2.2, 2.2],
       [2.2, 2.2, 2.2]])

print("Pointwise multiplication:\n", a * np.ones_like(a) * 2 )
print("Dot Product (matrix multiplication)\n", np.dot(a.T,
np.ones_like(a) * 2))

Pointwise multiplication:
 [[4.4 4.4 4.4]
 [4.4 4.4 4.4]]
Dot Product (matrix multiplication)
 [[8.8 8.8 8.8]
 [8.8 8.8 8.8]
 [8.8 8.8 8.8]]
```

Linear Algebra

numpy has implementations of some of the most common algebraic operations. The most common one, the dot product is implemented both as a method of the array object and as a numpy function.

Matrix multiplication can be performed either using the dot product or `np.matmul`. Using functions from numpy you can

- find eigenvalues and eigenvectors,
- compute some decompositions (QR, SVD),
- compute determinant of a matrix,
- norm of a vector,
- solve a linear system
- invert a matrix

See the documentation

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Almost all python plotting frameworks make use of matplotlib behind the scenes.

Plotting data is an effective way to convey the message hidden in the data. A good plot is worth a thousand words, and it can be difficult to unambiguously transmit your intended message. Different types of plots that represent the same data (i.e. scatter plots, linear plots, bar plots, pie charts etc.) can be perceived differently depending on who's looking.

Check out this article for some tips on better plotting:

Ten Simple Rules for Better Figures

NB: Do note that, regardless of the type of plot and data, ***all*** plots must have properly annotated axis ticks, axis labels, a title and a caption.

For a more comprehensive tutorial on matplotlib, check out this resource:

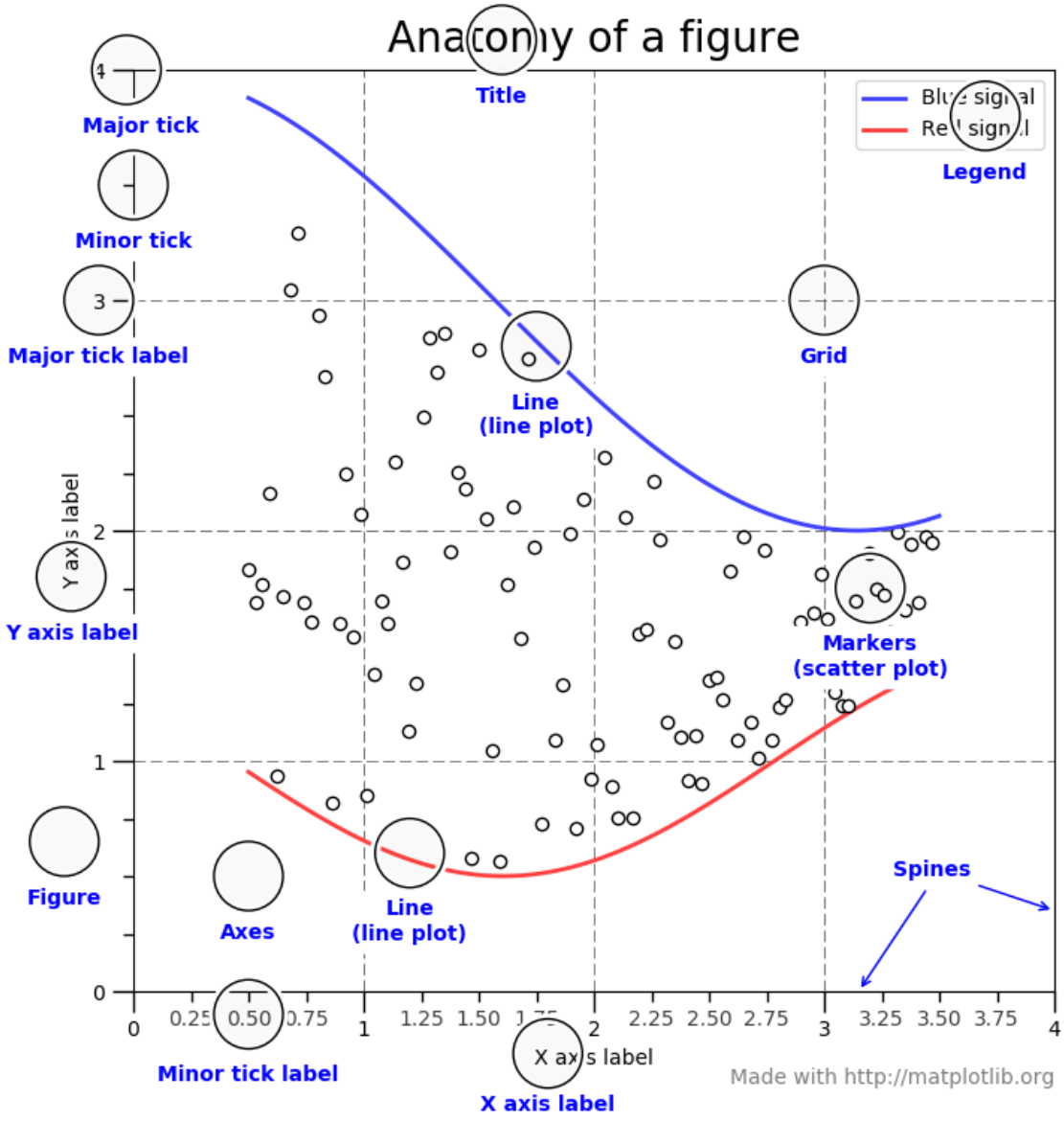
Matplotlib Tutorial – A Complete Guide to Python Plot w/ Examples

```
import matplotlib.pyplot as plt
%matplotlib inline
```

The elements of a plot

The figure and the axes

The figure is the entire image, the individual plots are called "axes". Here's a description from matplotlib's documentation.



Scatter & Line Plots

Scatter plots are used to plot data points on horizontal and vertical axis in the attempt to show how much one variable is affected by another. Each row in the data table is represented by a marker the position depends on its values in the columns set on the X and Y axes. A third variable can be set to correspond to the color or size of the markers, thus adding yet another dimension to the plot.

We will plot a simple regression line with generated data.

```
M = 10
N = 5

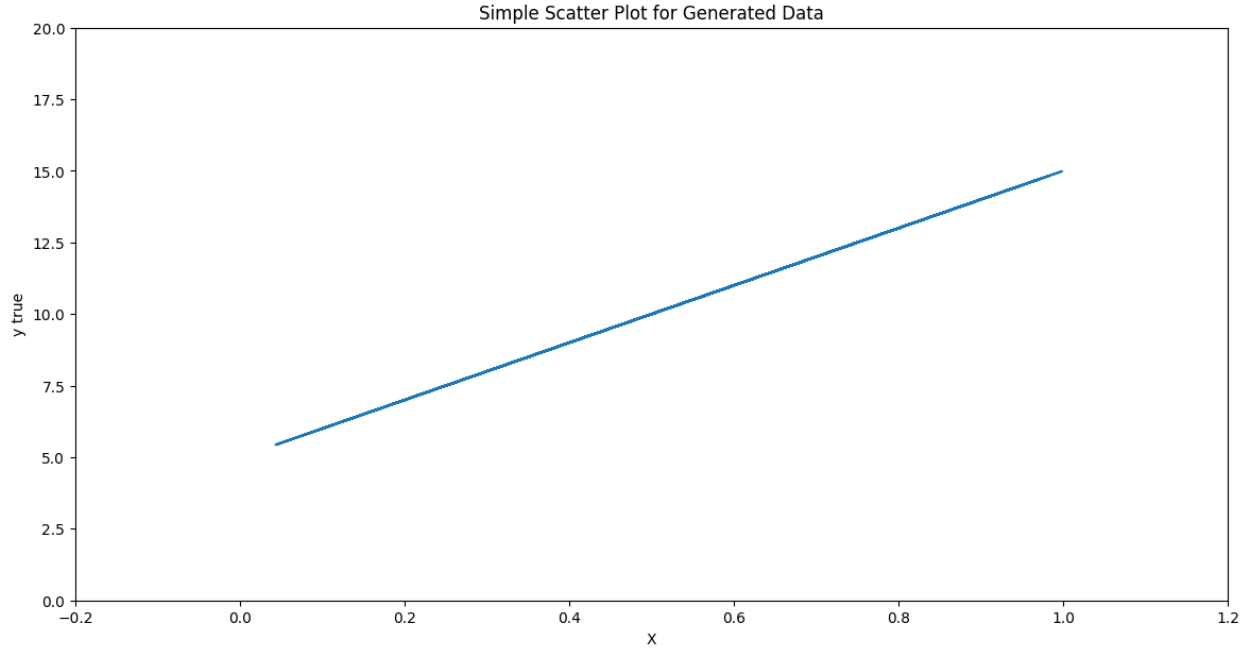
X = np.random.random(size = 100)
y_true = M * X + N

y_noise = y_true + np.random.normal(size = 100)

plt.plot(X, y_true)
plt.xlim(-0.2, 1.2)
plt.ylim(0, 20)

plt.xlabel("X")
plt.ylabel("y true")

plt.title("Simple Scatter Plot for Generated Data")
plt.gcf().set_size_inches(14, 7)
```

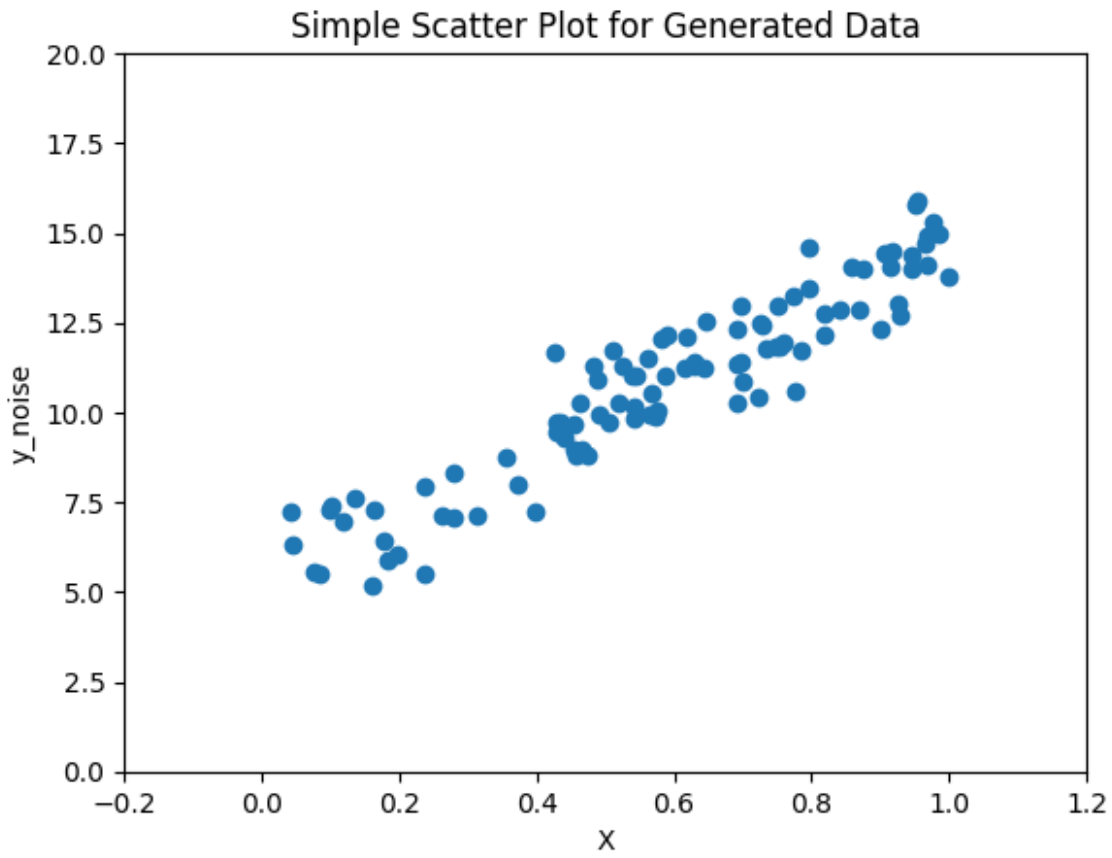



```
plt.scatter(X, y_noise)
plt.xlim(-0.2, 1.2)
plt.ylim(0, 20)

plt.xlabel("X")
plt.ylabel("y_noise")

plt.title("Simple Scatter Plot for Generated Data")
```

```
Text(0.5, 1.0, 'Simple Scatter Plot for Generated Data')
```



We can plot the two variables one on top of the other, for a better visualization. Make sure to add a legend, and different coloring, such that it is clear which part of the plot belongs to which set of variables.

Matplotlib also offers a wide range of **markers** to better differentiate between multiple data categories. Check https://matplotlib.org/api/markers_api.html for a full list.

```
plt.scatter(X, y_noise, label = 'noise', marker = '<')
plt.plot(X, y_true, color = 'r', label = 'true', linestyle = 'dashed',
linewidth = 1)

plt.xlim(-0.2, 1.2)
plt.ylim(0, 20)

plt.xlabel("X")
plt.ylabel("t")

plt.title("Simple Scatter Plot for Generated Data")
plt.legend()
plt.gcf().set_size_inches(14, 7)
```



Bar Plots

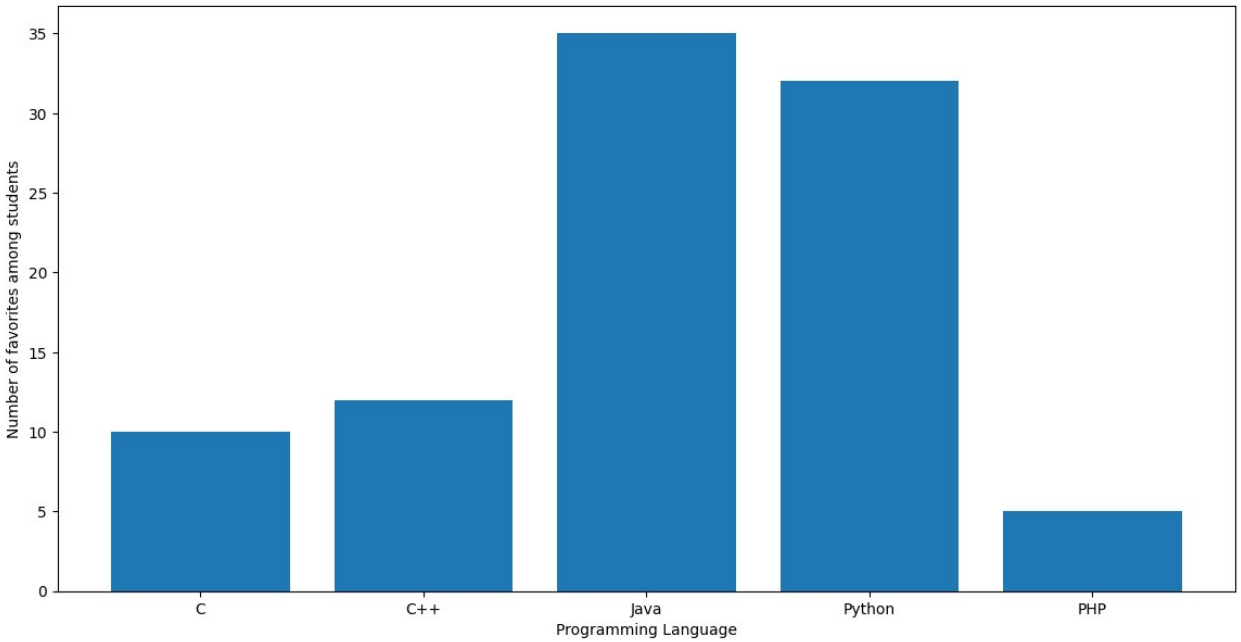
A bar chart or bar graph is a chart or graph that presents categorical data with rectangular bars with heights or lengths proportional to the values that they represent. The bars can be plotted vertically or horizontally.

A bar graph shows comparisons among discrete categories. One axis of the chart shows the specific categories being compared, and the other axis represents a measured value.

```
languages = ['C', 'C++', 'Java', 'Python', 'PHP']
students = [10, 12, 35, 32, 5]

ax = plt.subplot()
ax.bar(languages, students)

plt.xlabel('Programming Language')
plt.ylabel("Number of favorites among students")
plt.gcf().set_size_inches(14, 7)
```



We can make multiple bars for each category. It makes it easier for comparing different quantities with a common category.

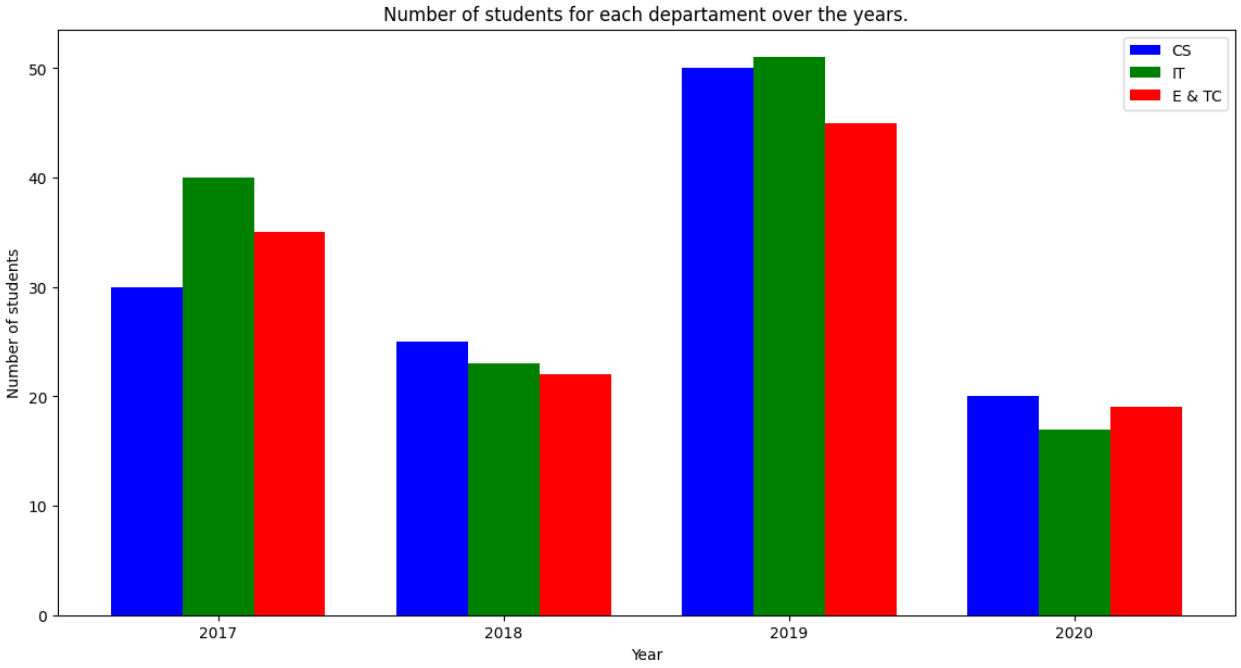
```
data = [
  [30, 25, 50, 20],
  [40, 23, 51, 17],
  [35, 22, 45, 19]
]

X = np.arange(4)
ax = plt.subplot()

ax.bar(X + 0.00, data[0], color = 'b', width = 0.25, label = 'CS')
ax.bar(X + 0.25, data[1], color = 'g', width = 0.25, label = 'IT')
ax.bar(X + 0.50, data[2], color = 'r', width = 0.25, label = 'E & TC')

ax.set_xticks(X + 0.25)
ax.set_xticklabels(X + 2017)

plt.xlabel("Year")
plt.ylabel("Number of students")
plt.title("Number of students for each departament over the years.")
ax.legend()
plt.gcf().set_size_inches(14, 7)
```



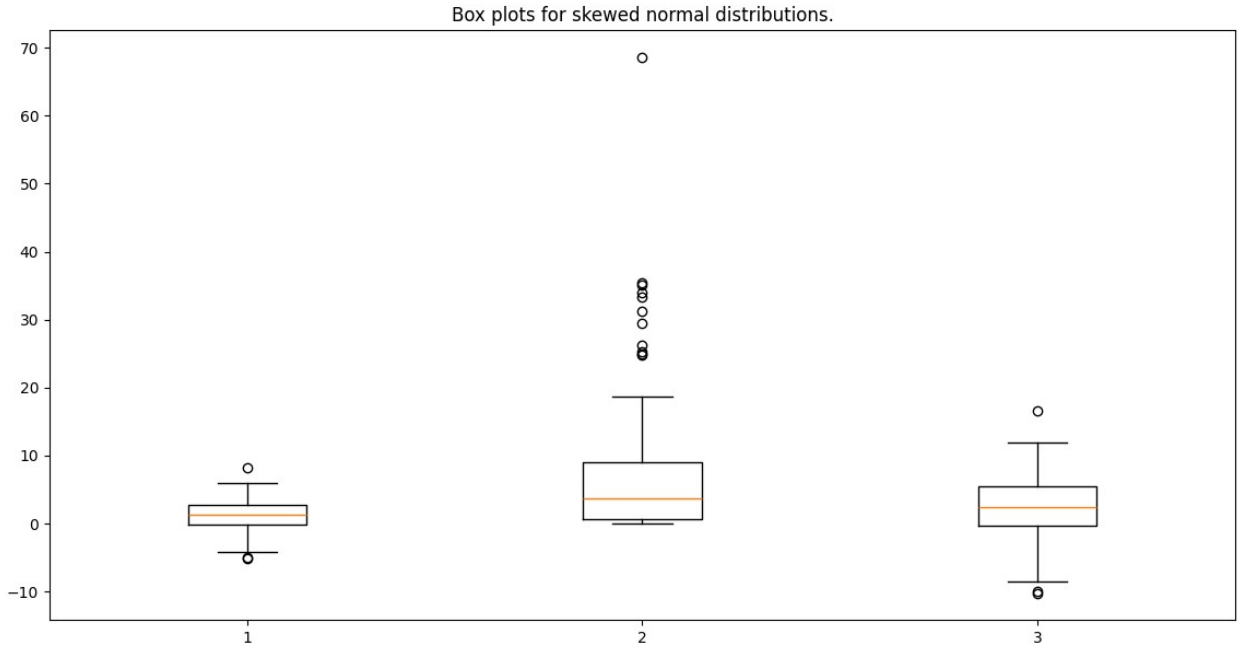
Box Plots

A boxplot is a standardized way of displaying the distribution of data based on a five number summary (“minimum”, first quartile (Q1), median, third quartile (Q3), and “maximum”). It can tell you about your outliers and what their values are. It can also tell you if your data is symmetrical, how tightly your data is grouped, and if and how your data is skewed.

```
data = np.random.normal(size = 100, scale = 3.0, loc = 1.0)

ax = plt.subplot()
ax.boxplot([data, data ** 2, data * 2])

plt.title("Box plots for skewed normal distributions.")
plt.gcf().set_size_inches(14, 7)
```



Exercises

After you finish your lab exercises, you should export this notebook as **pdf** and upload it to Moodle. (i.e. **File -> Print**, Destination: Save as PDF).

1. System of linear equations

Using numpy, solve the following linear equation:

$$\begin{bmatrix} 1 & 2 & 3.3 & 2 \\ 3 & 3.6 & 7 & 0 \\ 1 & 3 & -1 & 12 \\ 2 & 11 & 4 & 16 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 10 \\ 15 \\ 3 \\ -1 \end{bmatrix}$$

```
# Your code here
A = np.array([[1, 2, 3.3, 2], [3, 3.6, 7, 0], [1, 3, -1, 12], [2, 11,
4, 16]])
b = np.array([10, 15, 3, -1])
x = np.linalg.solve(A, b)
x
```

```
array([-1.28937298, -4.34942596,  4.9322932 ,  1.85582867])
```

2. Simple numpy exercises

1. Given an matrix, calculate the sum for each row

2. Given an vector, normalize the vector using the l_{10} norm

```
M = np.array([
    [1, 2, 3],
    [9, 3, 10],
    [3, 5, 0],
    [1, 7, -3],
    [0, -2, 3],

])

v = np.array([100, 2302, 2, -10, 134, -1])

# Your code here
from sklearn.preprocessing import normalize
res1 = np.zeros((M.shape[0], 1, 1), dtype=np.int_)
# vector de zerouri ce reprezinta suma fiecarui rand al matricei

for i in range(M.shape[0]):
  # iau fiecare rand
    sum = 0 # si calculez clasic suma
    for j in range(M.shape[1]):
        sum += M[i][j]
    res1[i] = sum # si-l pun in vector

l10_norm = np.linalg.norm(v, ord=10) # norma
res2 = v / l10_norm # se / la norma, ca sa se normalizeze

print("res1=", res1)
print("\nres2=", res2)
```

```
res1= [[[ 6]]

 [[22]]

 [[ 8]]

 [[ 5]]

 [[ 1]]]

res2= [ 4.34404865e-02  1.00000000e+00  8.68809731e-04 -4.34404865e-03
  5.82102520e-02 -4.34404865e-04]
```

3. Harder numpy exercises

Using the matrix and vector defined above:

1. Subtract the mean of each column in a given matrix.
2. Get the 3rd largest value in an array.

```
# Your code here

import statistics
(n, m) = M.shape # iau numarul de linii si de coloane

for j in range(m):
  column = M[:, j] # iau cate o coloana
  mean = statistics.mean(column) # ii fac media
  for i in range(column.size): # si ptr fiecare element al acelei
coloane scad media
    column[i] = column[i] - mean
  M[:, j] = column # si inlocuiesc in matricea de baza
print(M)

from heapq import nlargest

res4 = nlargest(3, v)[2]
# intoarce un vector de 3 chestii, indexat de la 0,
# cu elemente puse in ordine descrescatoare, de aceea iau [2]
res4
```

```
[[-1 -1  1]
 [ 7  0  8]
 [ 1  2 -2]
 [-1  4 -5]
 [-2 -5  1]]
```

```
100
```

4. Plot some functions

Using numpy and matplotlib, plot the following functions (you can choose the colors and styling, interval of interest and other parameters):

$$f_1(x) = max(0, x)$$

$$f_2(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

$$f_3(x) = \frac{1}{1 + e^{-x}}$$

$$f_4(x) = \frac{1}{\mu^2 + 1} * e^{-\frac{(x-\mu)^2}{b^2}}$$

$$f_5(x) = 0.5x\left(1 + tanh\left(0.797885\,x + 0.035677\,x^3\right)\right)$$

Note that all plots should have relevant ticks, labels and a title.

```
# Your code here
import math
import matplotlib.pyplot as plt
import numpy as np

# exemplu de date de intrare
x = np.linspace(-10, 10, 100)

# asemanator ca aici fac si la celelalte
# f1
xpoints = np.array(x) # vector
ypoints = list(map(lambda x: max(x, 0), x)) # folosesc functii lambda
plt.figure()
plt.plot(x, ypoints, label="f1")
# fac un chenar, pentru estetic
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
plt.grid()

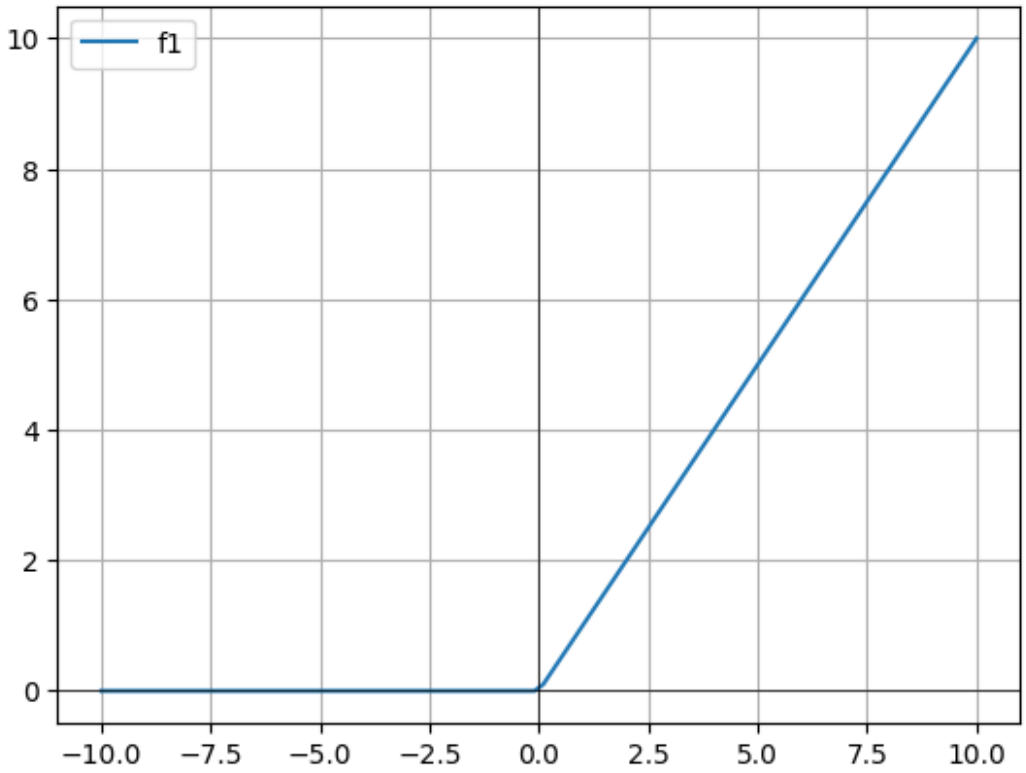
# f2
xpoints = np.array(x)
ypoints = list(map(lambda x: (np.exp(2*x) - 1)/(np.exp(2*x) + 1), x))
plt.figure()
plt.plot(x, ypoints, label="f2")
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
plt.grid()

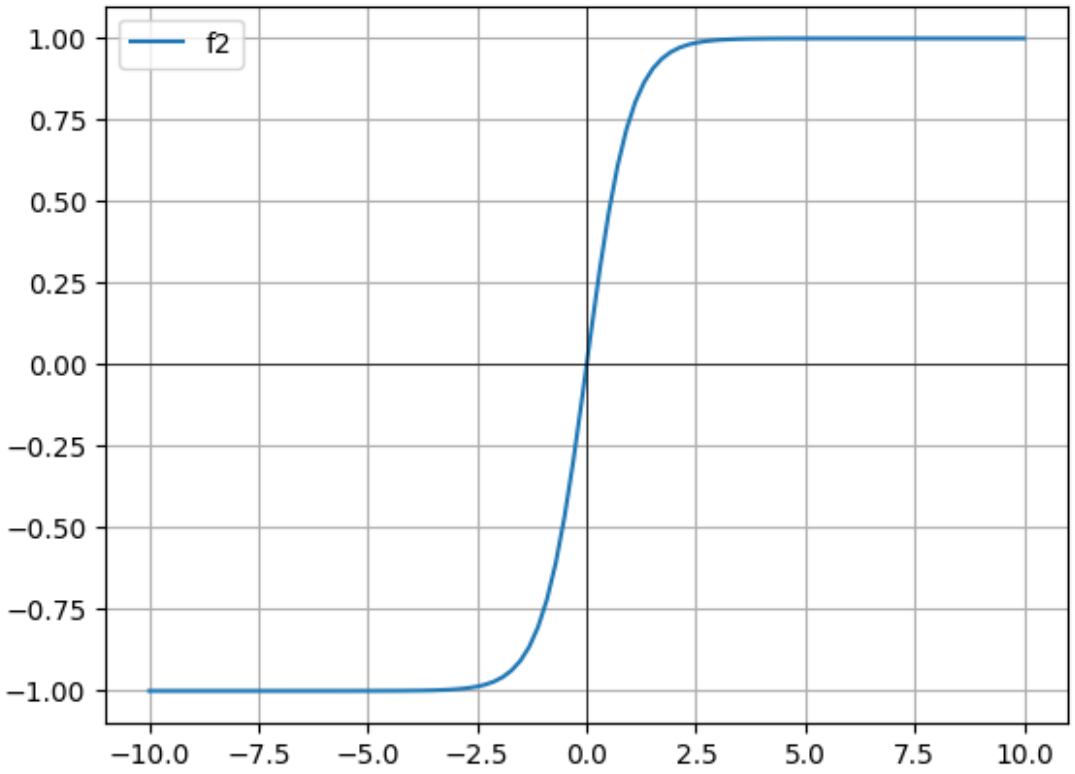
# f3
xpoints = np.array(x)
ypoints = list(map(lambda x: 1 /(1 + np.exp(-x)), x))
plt.figure()
plt.plot(x, ypoints, label="f3")
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
plt.grid()

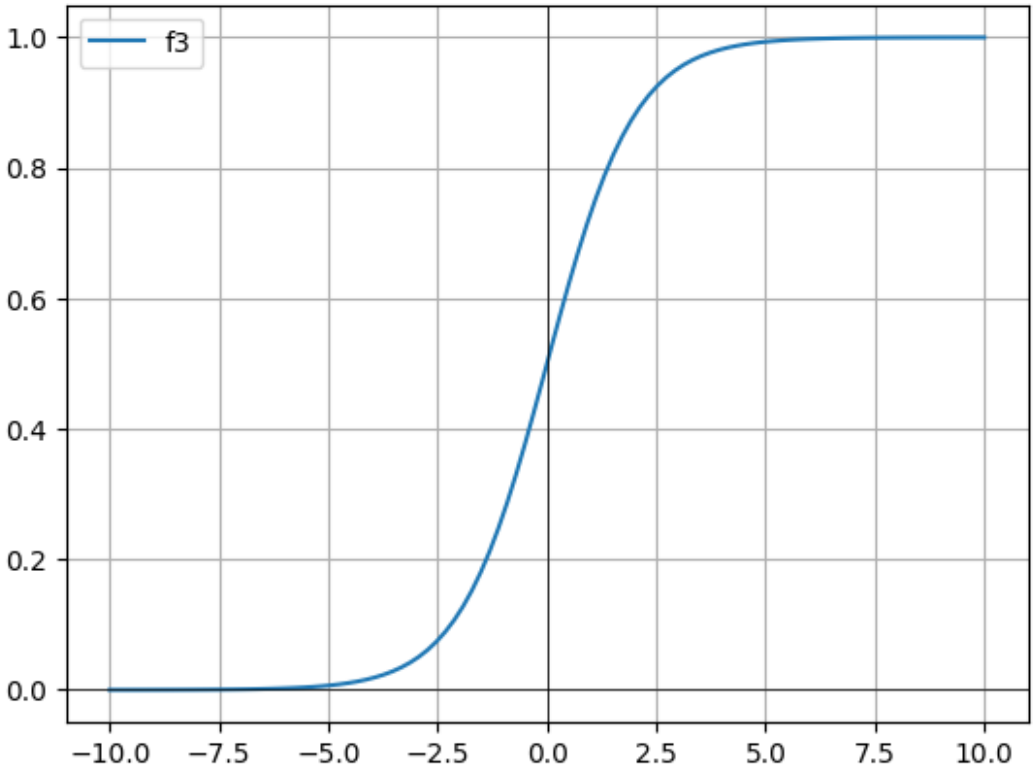
# f4
# nu stiu ce ar trebui sa fie asta
b = 1.5  # Valoare exemplu pentru b
# Calculez f4(x) cu mu = x
ypoints = list(map(lambda x: (1 / (x**2 + 1)) * np.exp(-((x - x)**2) /
(b**2)), x))
# Creez graficul
plt.figure(figsize=(8, 5))
plt.plot(x, ypoints, label="f4(x) cu mu = x")
plt.axhline(0, color='black', linewidth=0.5)
```

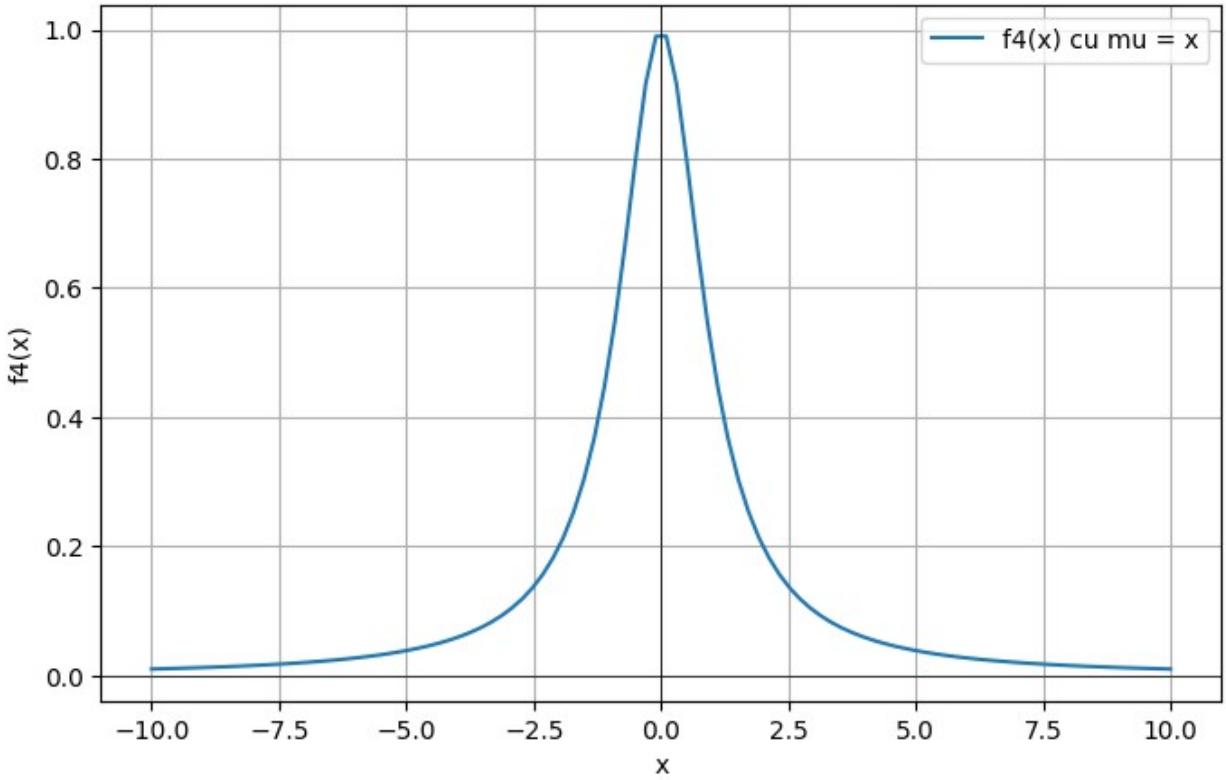
```
plt.axvline(0, color='black', linewidth=0.5)
plt.xlabel("x")
plt.ylabel("f4(x)")
plt.legend()
plt.grid()
plt.show()

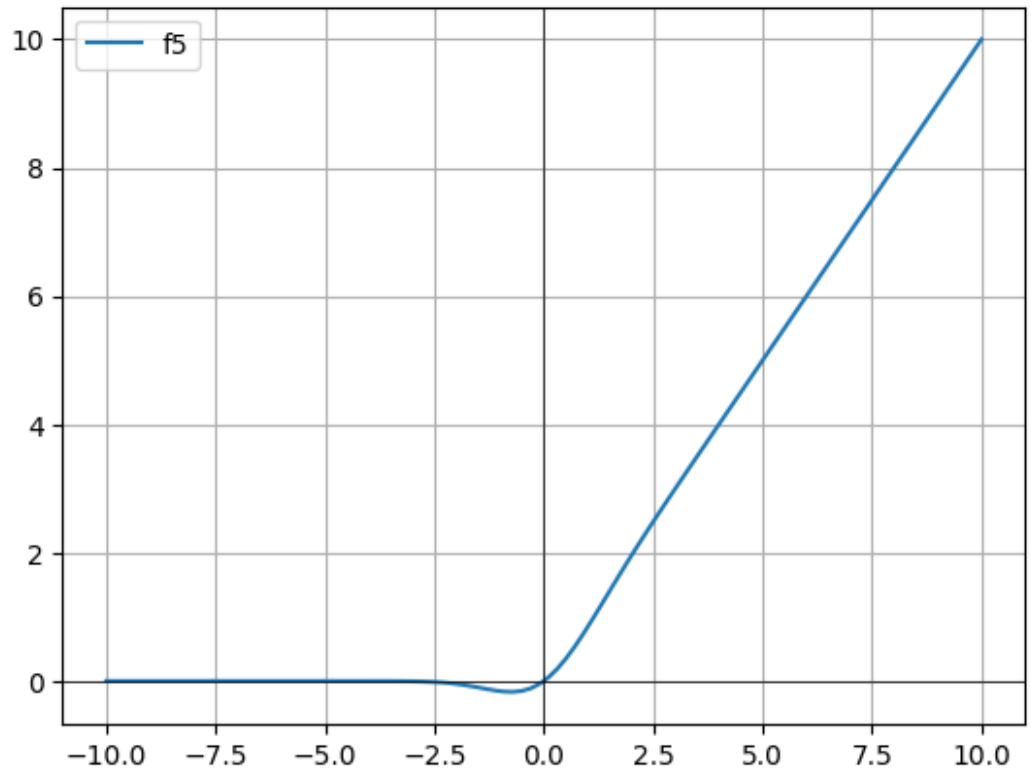
# f5
xpoints = np.array(x)
ypoints = list(map(lambda x: 0.5 * x * (1 + np.tanh(0.797885 *
x+0.035677 * x**3)), x))
plt.figure()
plt.plot(x, ypoints, label="f5")
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
plt.grid()
```




f2
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0


f3
1.0
0.8
0.6
0.4
0.2
0.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0


f4(x) cu mu = x
f4(x)
x
1.0
0.8
0.6
0.4
0.2
0.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0


f5
10
8
6
4
2
0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

5. Prettify Plot

Given the below plot, make it look presentable. Add **square** markers to scatter plots, **dashdotted** lines, add relevant axis **limits** and **ticks**, a **legend** and a **title**.

NB: A plot should be self-contained. Any reader should understand it without have to refer to its text description.

```
# Data Synthetization
x1 = np.linspace(0.0, 5.0)
x2 = np.linspace(0.0, 2.0)

y1 = np.cos(2 * np.pi * x1) * np.exp(-x1)
y2 = np.cos(2 * np.pi * x2)
y3 = np.cos(1 * np.pi * x1) * np.exp(-0.5 * x1) + 0.3 *
np.random.random(size = x1.shape)

# Plotting code
# TODO make this pretty.

fig, (ax1, ax2) = plt.subplots(2, 1)

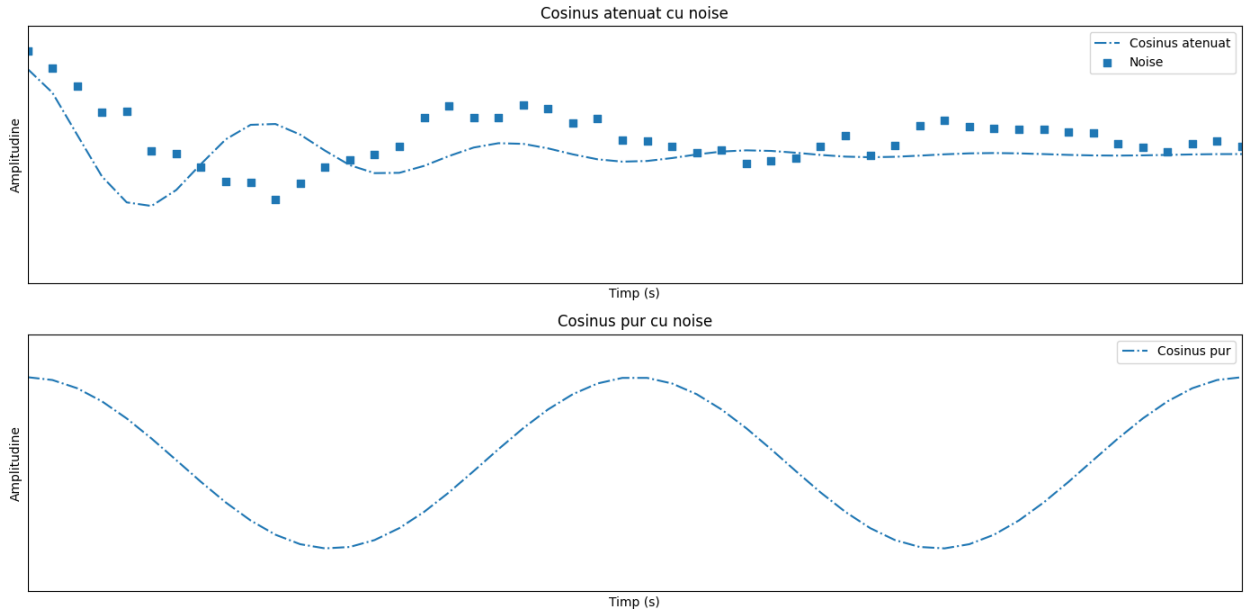
# ax1.plot(x1, y1) # ce era initial
ax1.plot(x1, y1, linestyle="dashdot",label="Cosinus atenuat")
# pun alt tip de linie si un titlu sa se inteleaga ce se intampla aici
# ax1.scatter(x1, y3) # ce era initial
ax1.scatter(x1, y3, marker="s", label="Noise")
# a zis sa pun si markeri
# hai sa pun niste limite, sa fac zoom in
ax1.set_xlim(0, 5)
ax1.set_ylim(-1.5, 1.5)
# niste etichete pentru referinta
ax1.set_xlabel("Timp (s)")
ax1.set_ylabel("Amplitudine")
# astea cred ca erau deja
ax1.get_xaxis().set_ticks([])
ax1.get_yaxis().set_ticks([])

# niste etichete pentru referinta
ax1.set_title("Cosinus atenuat cu noise")
ax1.legend()
ax1.grid(True)

# asemanator ca mai sus
# ax2.plot(x2, y2)
ax2.plot(x2, y2, linestyle="dashdot",label="Cosinus pur")
ax2.set_xlim(0, 2)
ax2.set_ylim(-1.5, 1.5)
ax2.set_xlabel("Timp (s)")
ax2.set_ylabel("Amplitudine")
```

```
ax2.get_xaxis().set_ticks([])
ax2.get_yaxis().set_ticks([])
ax2.set_title("Cosinus pur cu noise")
ax2.legend()
ax2.grid(True)

plt.gcf().set_size_inches(14, 7)
# Adjust layout
plt.tight_layout()
plt.show()
```



6. Plotting a dataset

Plot different aspects of the Wine Dataset from sklearn. These data are the results of a chemical analysis of wines grown in the same region in Italy but derived from three different cultivars.

6.1 Create a plot for visualizing the distribution of one features (e.g. alchohol level / malic acid / ash etc.) for each class of wine.

6.2 Create 3 subplots highlighting different aspects of the dataset. Choose the appropriate plot types (scatter plot, line plot, bar plot, box plot etc.). You can choose various combinations of columns to gain a better understanding of this dataset.

Be creative! You can color your points using the `target` array. Remember to always have labels on your axes, appropriate ticks, a legend where necessary, and a plot title.

```
from sklearn.datasets import load_wine

wine_dataset = load_wine()
```

```
data = wine_dataset['data']
target = wine_dataset['target']

column_names = wine_dataset['feature_names']

data

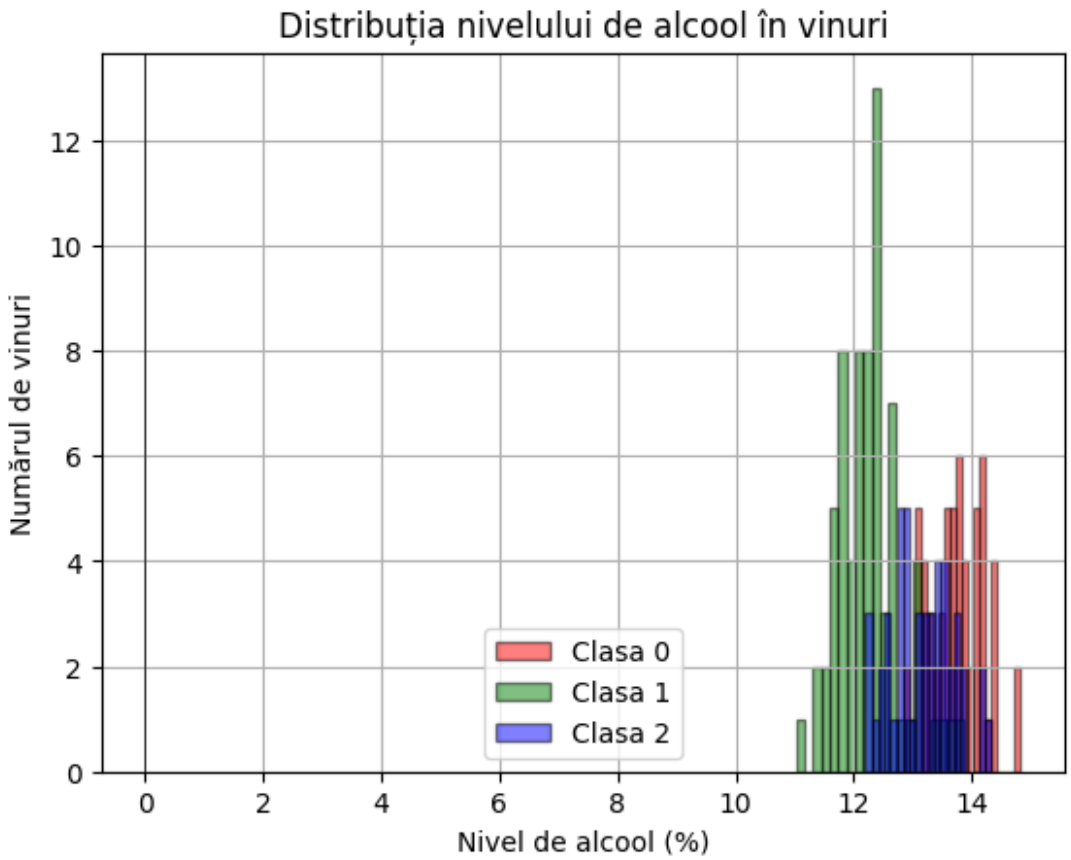
# TODO your code here
# Selectez prima caracteristică (nivelul de alcool) (cf a ce m-am
documentat)
alcool = data[:, 0]
culori = ['red', 'green', 'blue']

# Creez graficul
plt.figure()
plt.xlabel("Nivel de alcool (%)")
plt.ylabel("Numărul de vinuri")
plt.title("Distribuția nivelului de alcool în vinuri")

# fac histograma pentru fiecare clasă de vinuri -> am vazut ca sunt 3
for i in range(3):
    plt.hist(alcool[target == i], bins=20, alpha=0.5, color=culori[i],
edgecolor='black', label=f"Clasa {i}")

# prettify
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)

plt.legend()
plt.grid()
plt.show()
```


Distribuția nivelului de alcool în vinuri
Numărul de vinuri
Nivel de alcool (%)
Clasa 0
Clasa 1
Clasa 2
0
2
4
6
8
10
12
14